LAPORAN PRAKTIKUM PERTEMUAN 5



Dosen Pengampu: Juni Ismail, S.Kom., M.Kom.

Disusun Oleh :

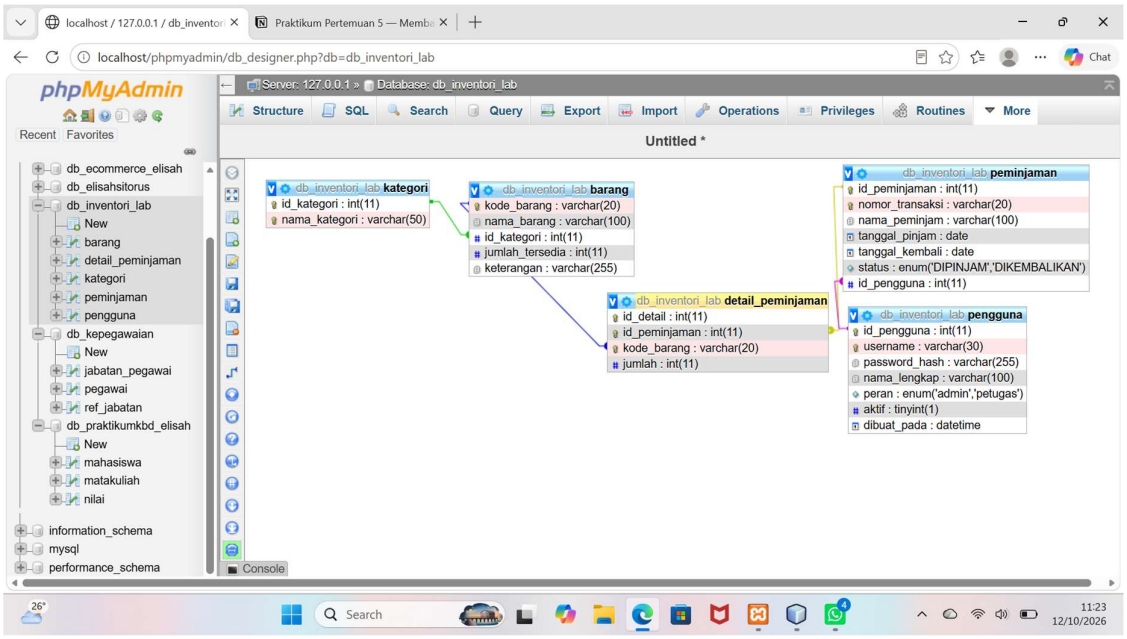
Nama : Daniel Sianipar
Nim : 25110104
Prodi : Teknik Komputer

A. Tujuan dan Gambaran Umum

Praktikum ini dilakukan untuk membuat dan menguji database inventori laboratorium bernama db_inventori_lab melalui MySQL dengan bantuan phpMyAdmin. Laporan berisi screenshot dan uraian setiap query SQL, meliputi desain relasi tabel, pengambilan data (SELECT, WHERE, LIKE, JOIN, GROUP BY), pengelolaan data (INSERT, UPDATE, DELETE), serta pengujian aturan integritas seperti primary key, foreign key, NOT NULL, dan CHECK yang sengaja dibuat tidak valid.

1. Desain Database dan Relasi Antartabel (Designer)

Bagian ini memperlihatkan fitur Designer pada phpMyAdmin untuk database db_inventori_lab. Melalui tampilan tersebut, struktur tabel, nama kolom, tipe data, dan relasi foreign key antar tabel dapat dilihat dengan jelas.



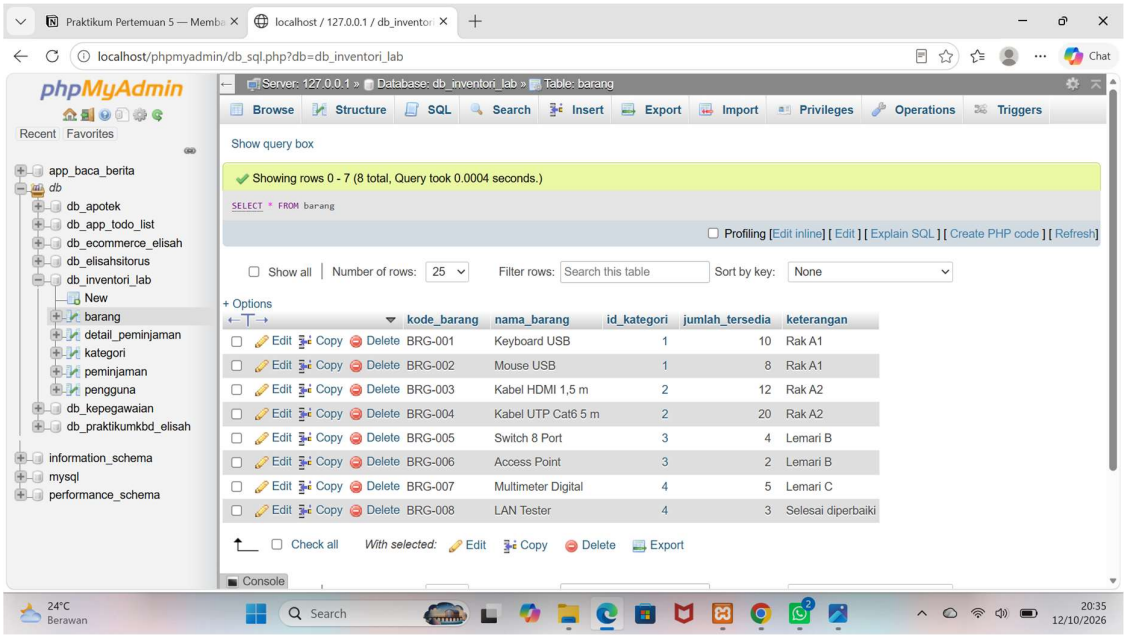
Hasil: Struktur database telah memisahkan data kategori, barang, pengguna/petugas, dan transaksi ke tabel yang berbeda. Hubungan primary key dan foreign key membantu menjaga keteraturan serta konsistensi data.

2. Menampilkan Seluruh Data Barang

Query berikut digunakan untuk menampilkan seluruh kolom dan baris yang tersimpan pada tabel barang. Simbol * menunjukkan bahwa semua kolom akan ditampilkan.

Perintah SQL:

```
SELECT * FROM barang;
```



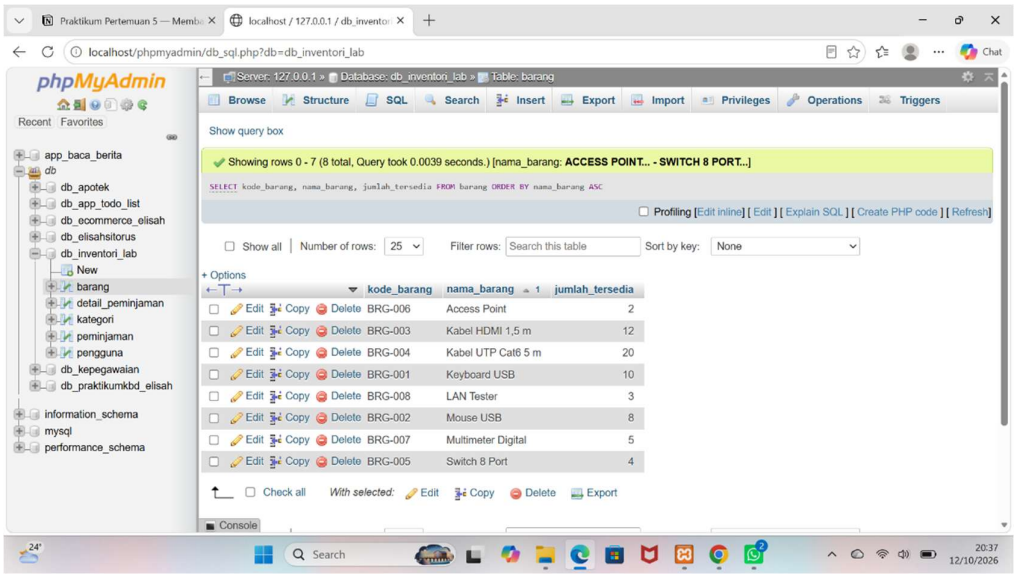
Hasil: Perintah SELECT * menampilkan isi tabel secara lengkap sehingga data yang sudah dimasukkan dapat diperiksa.

3. Memilih Kolom Tertentu dan Mengurutkan Data

Query ini mengambil hanya kolom kode_barang, nama_barang, dan jumlah_tersedia dari tabel barang. Data kemudian diurutkan menurut nama barang secara menaik.

Perintah SQL:

```
SELECT kode_barang, nama_barang, jumlah_tersedia
FROM barang
ORDER BY nama_barang ASC;
```



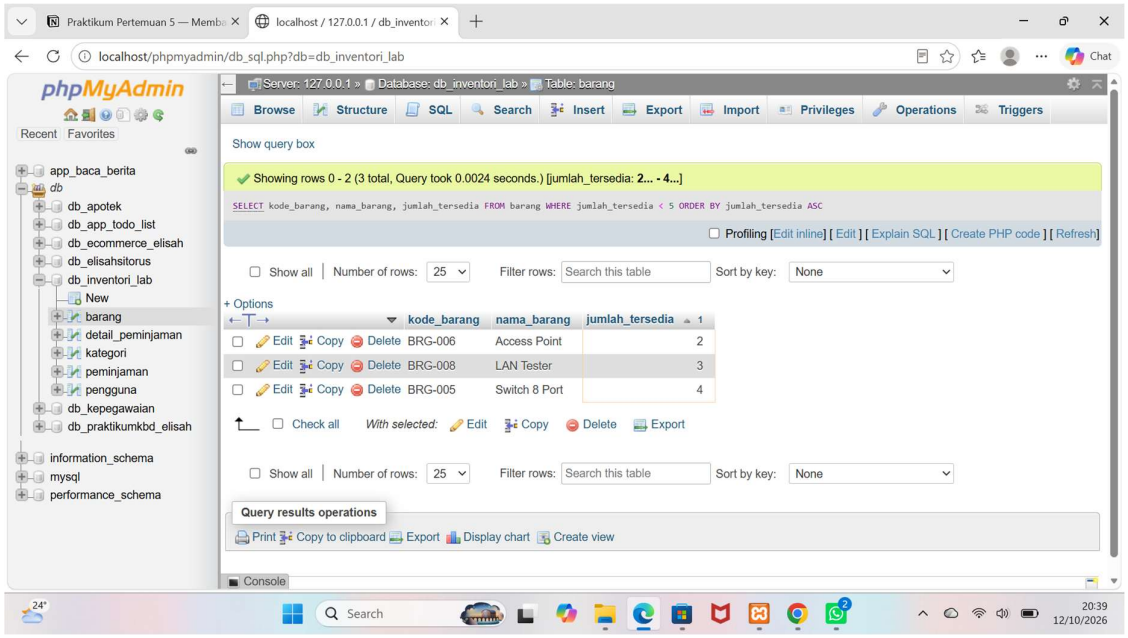
Hasil: Sebanyak 8 baris ditampilkan dan urutannya mengikuti abjad nama barang.

4. Menyaring Data dengan WHERE (Stok Menipis)

Query ini digunakan untuk mencari barang dengan persediaan kurang dari 5 unit. Hasilnya diurutkan mulai dari jumlah stok yang paling rendah.

Perintah SQL:

```
SELECT kode_barang, nama_barang, jumlah_tersedia
FROM barang
WHERE jumlah_tersedia < 5
ORDER BY jumlah_tersedia ASC;
```



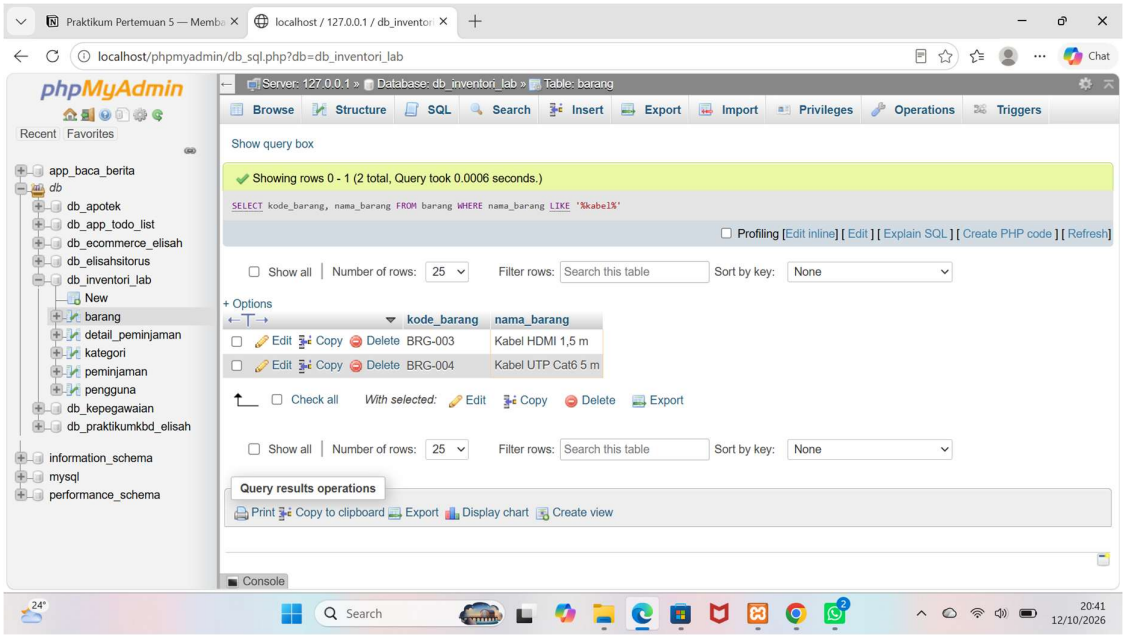
Hasil: Ditemukan 3 barang dengan stok di bawah 5, yaitu Access Point (BRG-006) sebanyak 2 unit, LAN Tester (BRG-008) sebanyak 3 unit, dan Switch 8 Port (BRG-005) sebanyak 4 unit. Barang-barang tersebut perlu diperhatikan untuk pengadaan atau penanganan lebih lanjut.

5. Pencarian Teks dengan LIKE

Perintah LIKE berikut mencari nama barang yang memiliki kata “kabel”, baik di awal, tengah, maupun akhir teks.

Perintah SQL:

```
SELECT kode_barang, nama_barang
FROM barang
WHERE nama_barang LIKE '%kabel%';
```



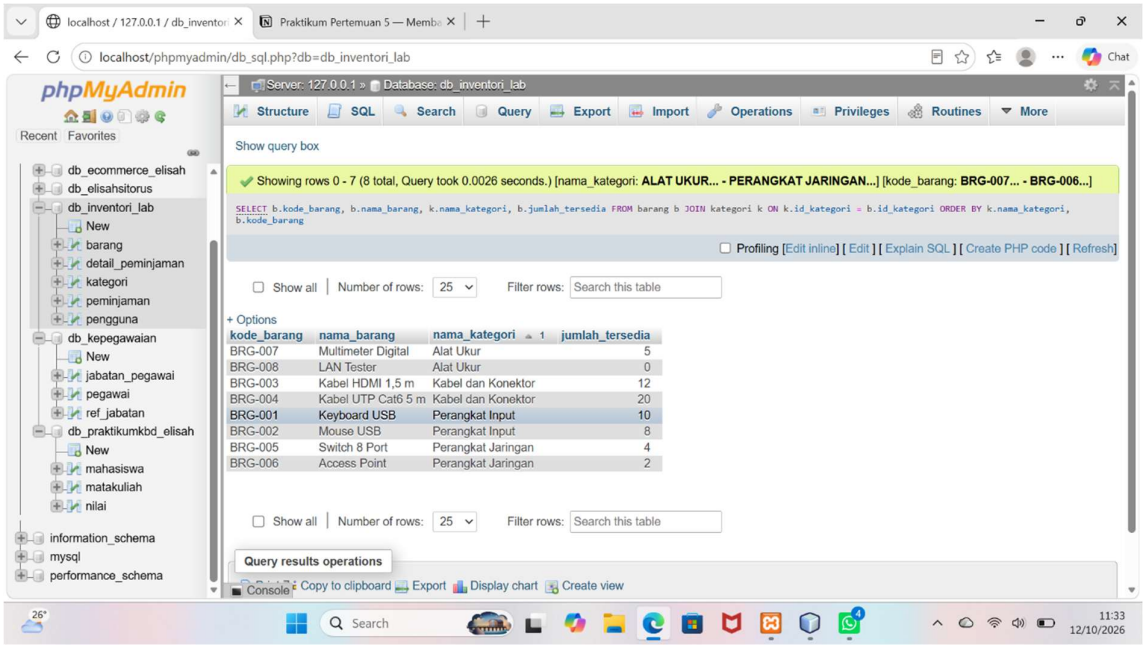
Hasil: Dua barang yang cocok ditemukan, yaitu Kabel HDMI 1,5 m (BRG-003) dan Kabel UTP Cat6 5 m (BRG-004).

6. Menggabungkan Dua Tabel dengan JOIN (barang dan kategori)

Pada tabel barang, informasi kategori disimpan sebagai nilai id_kategori. JOIN menghubungkan nilai tersebut dengan tabel kategori agar nama kategorinya ikut ditampilkan.

Perintah SQL:

```
SELECT b.kode_barang, b.nama_barang, k.nama_kategori, b.jumlah_tersedia
FROM barang b
JOIN kategori k ON k.id_kategori = b.id_kategori
ORDER BY k.nama_kategori, b.kode_barang;
```



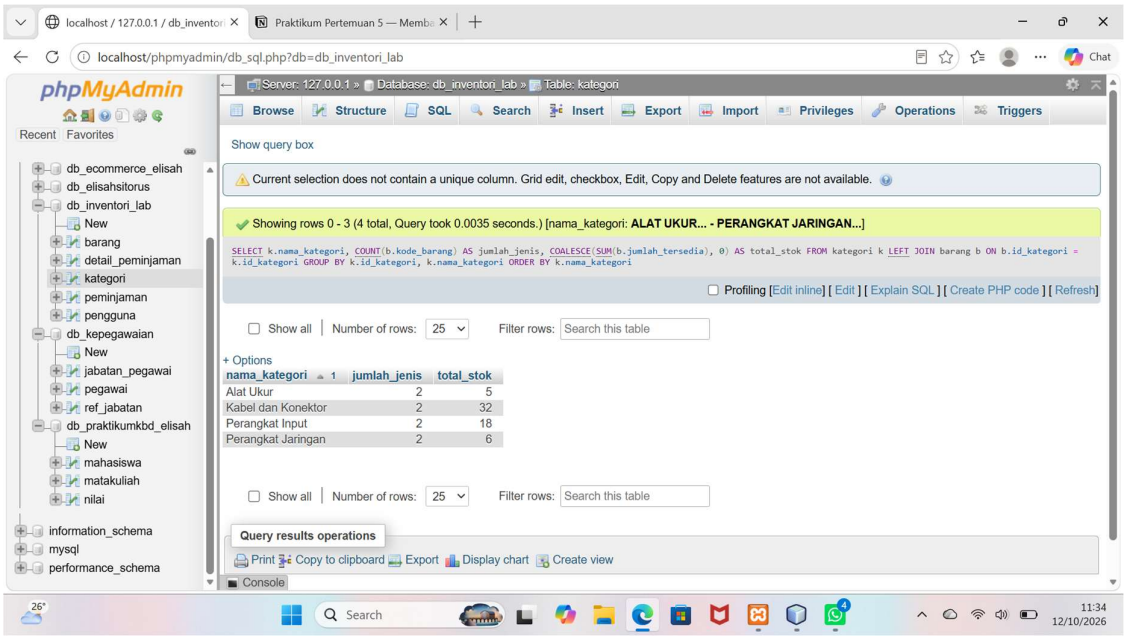
Hasil: Query menghasilkan 8 baris yang dilengkapi nama kategori, sehingga informasi lebih mudah dipahami dibandingkan jika hanya menampilkan id_kategori.

7. Rekap per Kategori dengan LEFT JOIN, COUNT, SUM, dan GROUP BY

Query ini menyusun ringkasan untuk setiap kategori, berupa jumlah jenis barang dan total seluruh stok yang tersedia.

Perintah SQL:

```
SELECT k.nama_kategori,
       COUNT(b.kode_barang) AS jumlah_jenis,
       COALESCE(SUM(b.jumlah_tersedia), 0) AS total_stok
FROM kategori k
LEFT JOIN barang b ON b.id_kategori = k.id_kategori
GROUP BY k.id_kategori, k.nama_kategori
ORDER BY k.nama_kategori;
```



Hasil: Rekap menunjukkan 4 kategori: Alat Ukur memiliki 2 jenis dengan stok 5, Kabel dan Konektor memiliki 2 jenis dengan stok 32, Perangkat Input memiliki 2 jenis dengan stok 18, serta Perangkat Jaringan memiliki 2 jenis dengan stok 6.

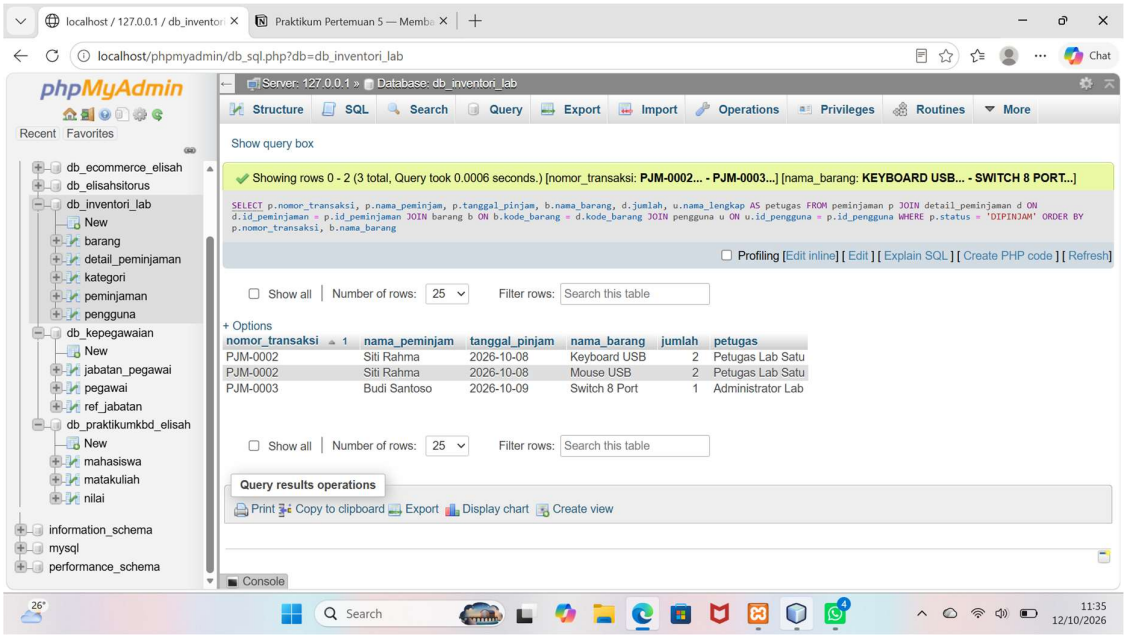
8. Laporan Peminjaman Aktif (JOIN Empat Tabel)

Query ini menggabungkan tabel peminjaman, detail_peminjaman, barang, dan pengguna untuk menampilkan informasi transaksi yang statusnya masih DIPINJAM.

Perintah SQL:

```
SELECT p.nomor_transaksi, p.nama_peminjam, p.tanggal_pinjam,
       b.nama_barang, d.jumlah, u.nama_lengkap AS petugas
FROM peminjaman p
JOIN detail_peminjaman d ON d.id_peminjaman = p.id_peminjaman
JOIN barang b ON b.kode_barang = d.kode_barang
```

```
JOIN pengguna u ON u.id_pengguna = p.id_pengguna
WHERE p.status = 'DIPINJAM'
ORDER BY p.nomor_transaksi, b.nama_barang;
```



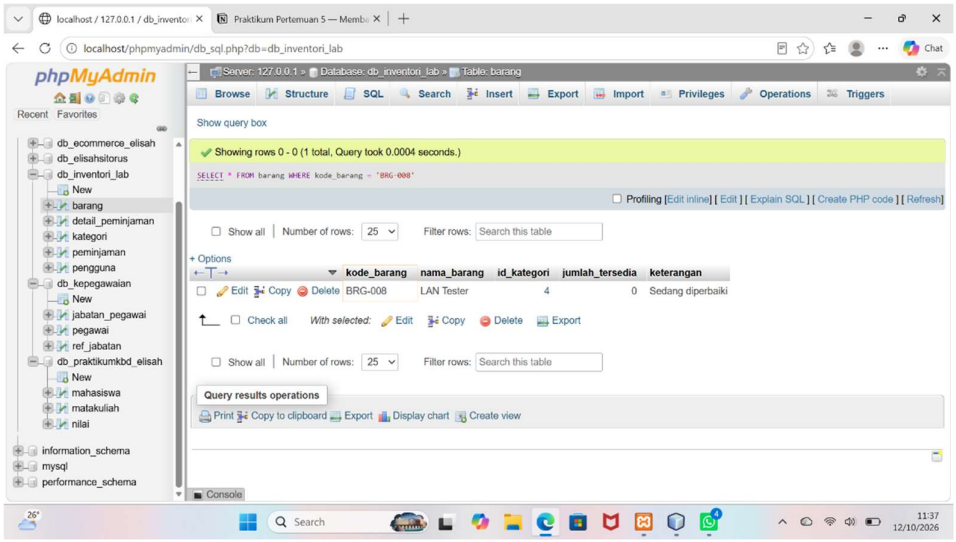
Hasil: Terdapat 3 baris data. Transaksi PJM-0002 milik Siti Rahma pada 2026-10-08 mencakup Keyboard USB sebanyak 2 unit dan Mouse USB sebanyak 2 unit, dengan layanan Petugas Lab Satu. Transaksi PJM-0003 milik Budi Santoso pada 2026-10-09 mencakup Switch 8 Port sebanyak 1 unit dan dilayani Administrator Lab.

9. Memeriksa Data Sebelum UPDATE

Sebelum menjalankan UPDATE, data dengan kode barang yang dituju ditampilkan terlebih dahulu untuk memastikan baris yang akan diubah sudah benar.

Perintah SQL:

```
SELECT * FROM barang
WHERE kode_barang = 'BRG-008';
```



Hasil:

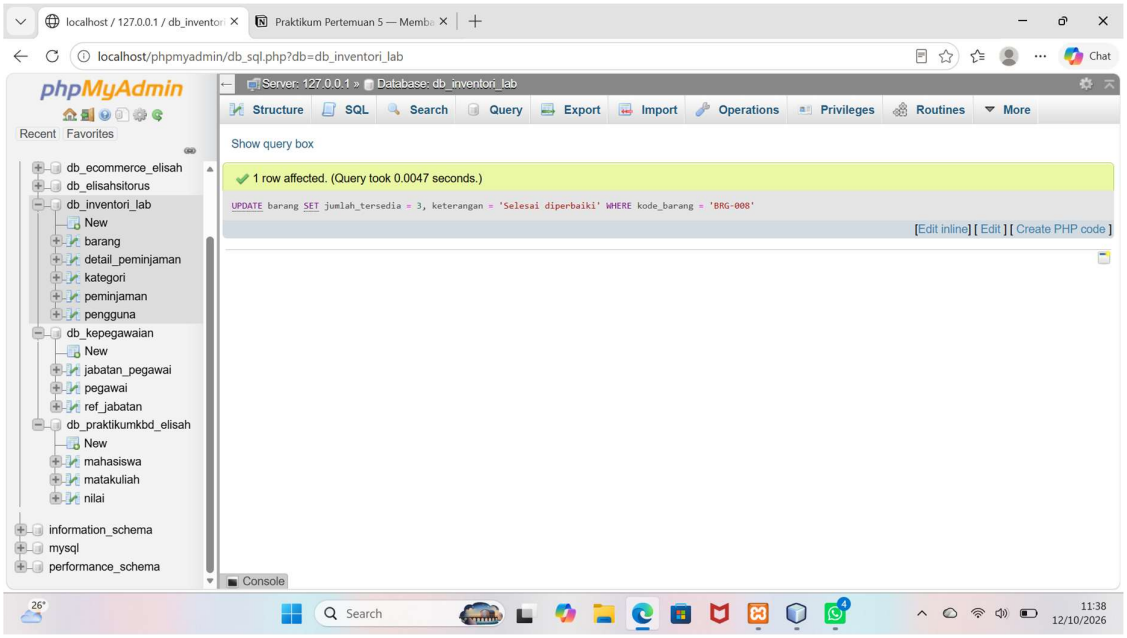
Tampilan data sebelum perubahan menjadi acuan untuk membandingkan kondisi setelah query UPDATE dijalankan.

10. Mengubah Data dengan UPDATE

UPDATE berfungsi mengubah nilai pada data yang sudah tersimpan. Pada contoh ini, LAN Tester yang telah selesai diperbaiki diperbarui jumlah stok dan keterangannya.

Perintah SQL:

```
UPDATE barang
SET jumlah_tersedia = 3,
    keterangan = 'Selesai diperbaiki'
WHERE kode_barang = 'BRG-008';
```



Hasil: Informasi barang dengan kode BRG-008 telah diperbarui. Perubahannya dapat diperiksa pada tampilan tabel barang.

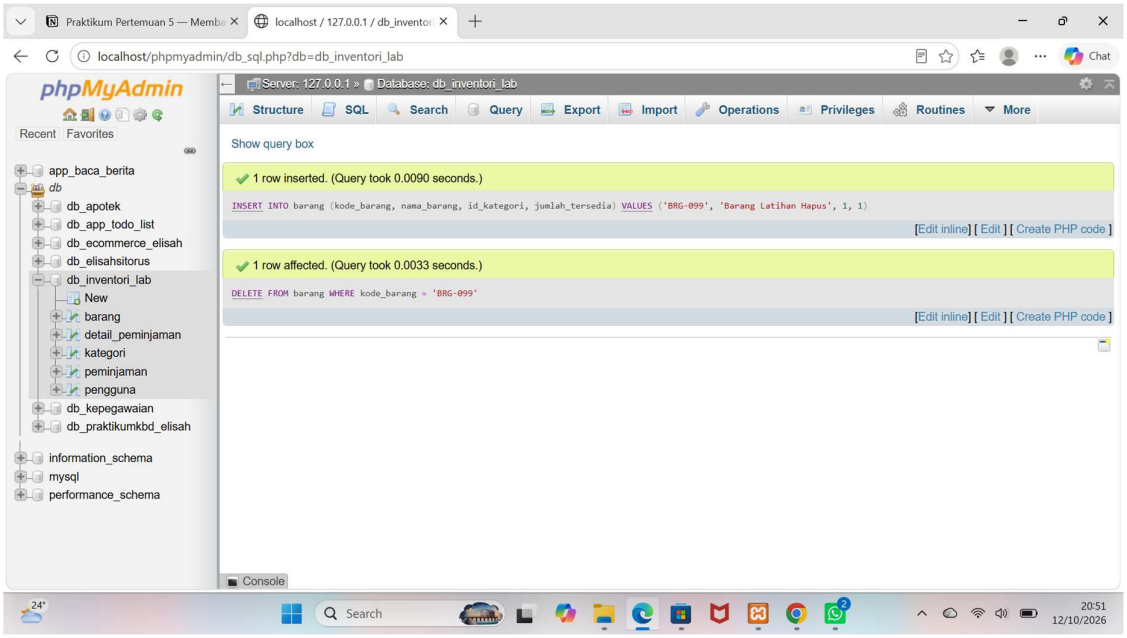
11. Menambah Data dengan INSERT dan Menghapus dengan DELETE

Langkah ini menjalankan dua operasi secara berurutan, yaitu memasukkan satu data barang percobaan dan kemudian menghapus data tersebut.

Perintah SQL:

```
INSERT INTO barang (kode_barang, nama_barang, id_kategori,
jumlah_tersedia)
VALUES ('BRG-099', 'Barang Latihan Hapus', 1, 1);

DELETE FROM barang
WHERE kode_barang = 'BRG-099';
```



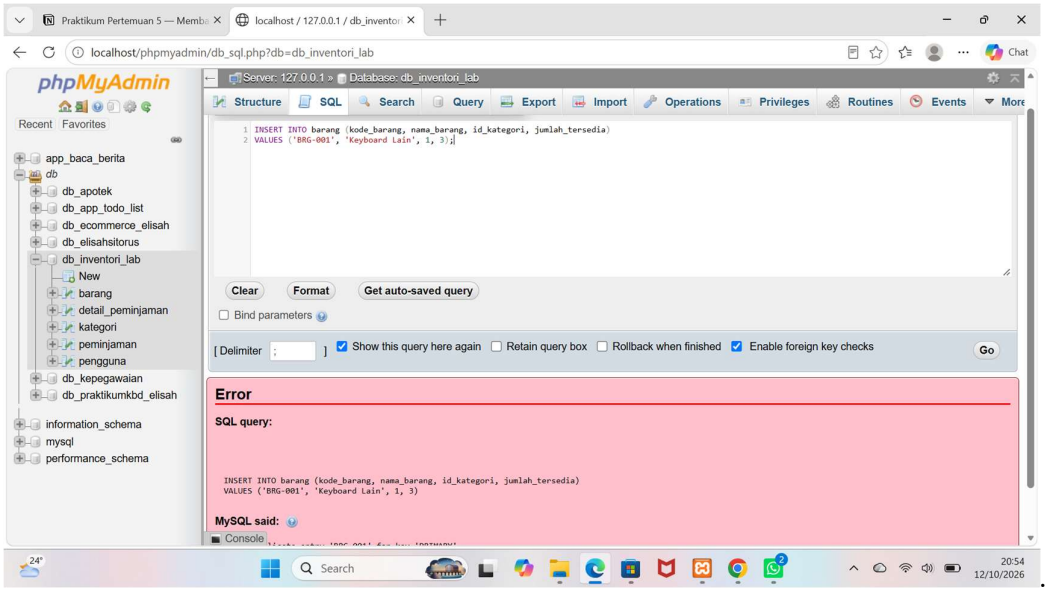
Hasil: Data percobaan berhasil ditambahkan lalu dihapus kembali, sehingga isi tabel barang kembali ke keadaan semula.

12. Uji Error 1: Primary Key Duplikat

Pengujian ini mencoba memasukkan kode barang BRG-001 yang sudah digunakan. Tujuannya untuk memastikan aturan primary key menolak kode yang sama.

Perintah SQL:

```
INSERT INTO barang (kode_barang, nama_barang, id_kategori,
jumlah_tersedia)
VALUES ('BRG-001', 'Keyboard Lain', 1, 3);
```



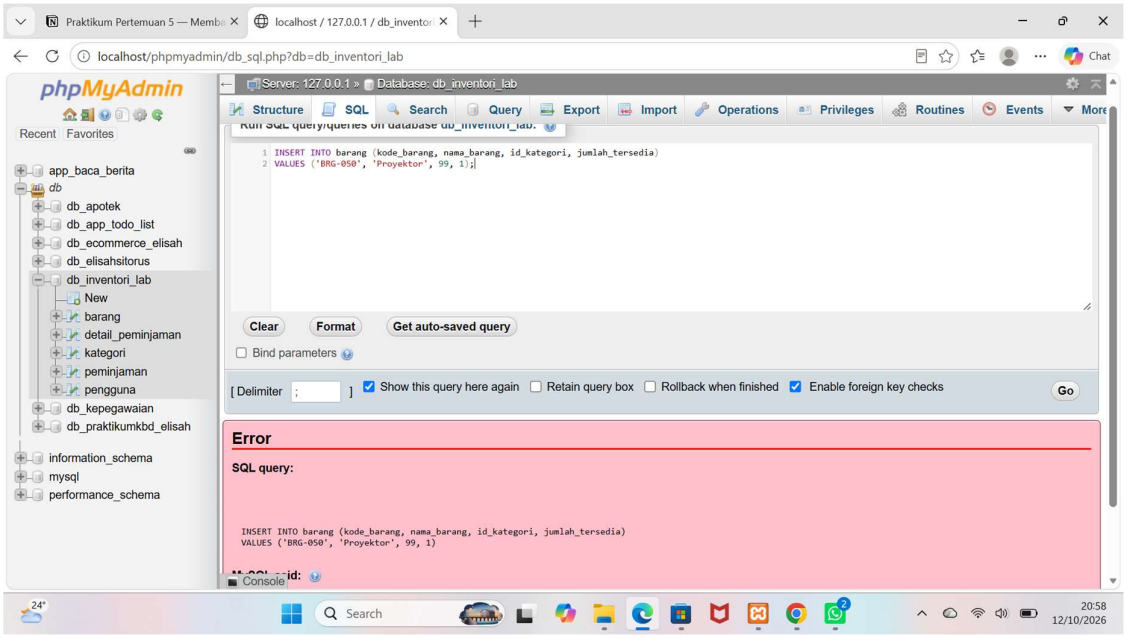
Hasil: Query ditolak dan data tabel tidak berubah. Primary key memastikan setiap barang memiliki kode unik.

13. Uji Error 2: Pelanggaran Foreign Key (Kategori Tidak Ada)

Percobaan ini menambahkan barang dengan id_kategori 99, sedangkan kategori dengan ID tersebut tidak tersedia pada tabel kategori.

Perintah SQL:

```
INSERT INTO barang (kode_barang, nama_barang, id_kategori,
jumlah_tersedia)
VALUES ('BRG-050', 'Proyektor', 99, 1);
```



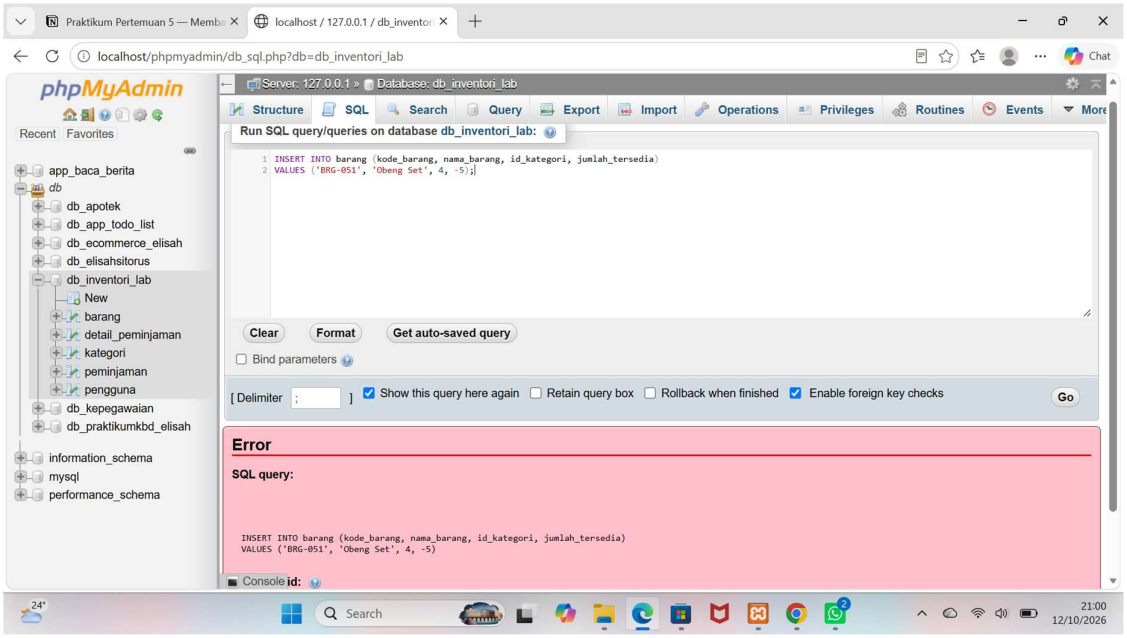
Hasil: Query gagal dijalankan karena foreign key mengharuskan setiap id_kategori merujuk pada kategori yang benar-benar ada.

14. Uji Error 3: Pelanggaran CHECK Constraint (Stok Negatif)

Pada pengujian ini, jumlah stok diisi dengan angka negatif -5 untuk memastikan sistem menolak nilai persediaan yang tidak valid.

Perintah SQL:

```
INSERT INTO barang (kode_barang, nama_barang, id_kategori,
jumlah_tersedia)
VALUES ('BRG-051', 'Obeng Set', 4, -5);
```



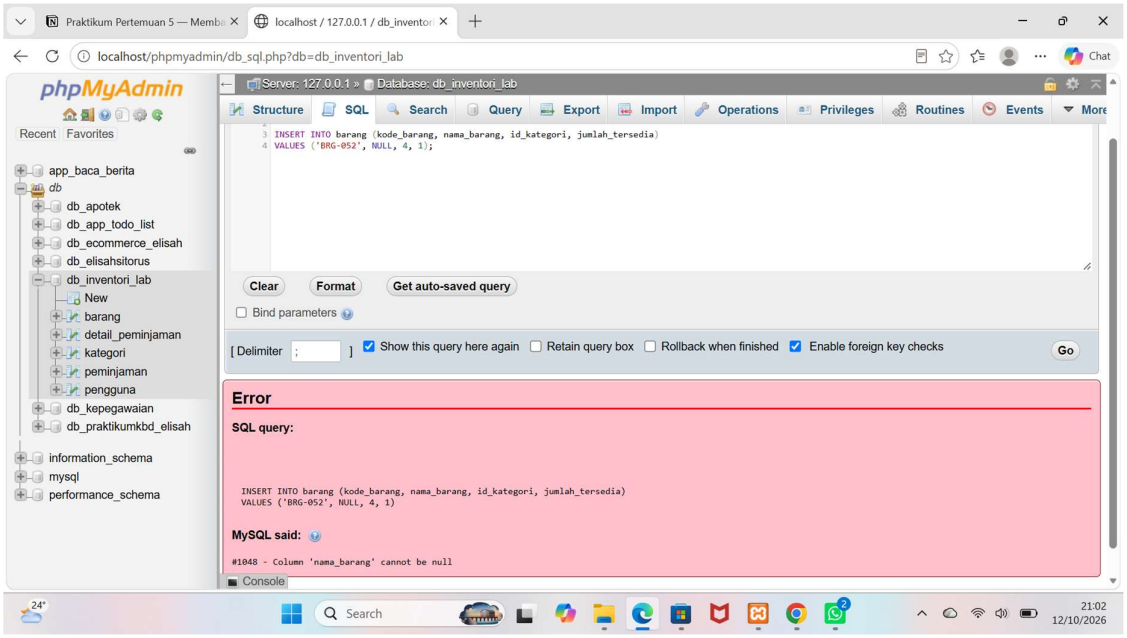
Hasil: Query tidak berhasil karena CHECK constraint membatasi stok agar memenuhi aturan nilai yang ditentukan.

15. Uji Error 4: Kolom NOT NULL Diisi NULL

Pengujian ini mencoba menyimpan data barang dengan kolom nama_barang bernilai NULL.

Perintah SQL:

```
INSERT INTO barang (kode_barang, nama_barang, id_kategori,
jumlah_tersedia)
VALUES ('BRG-052', NULL, 4, 1);
```



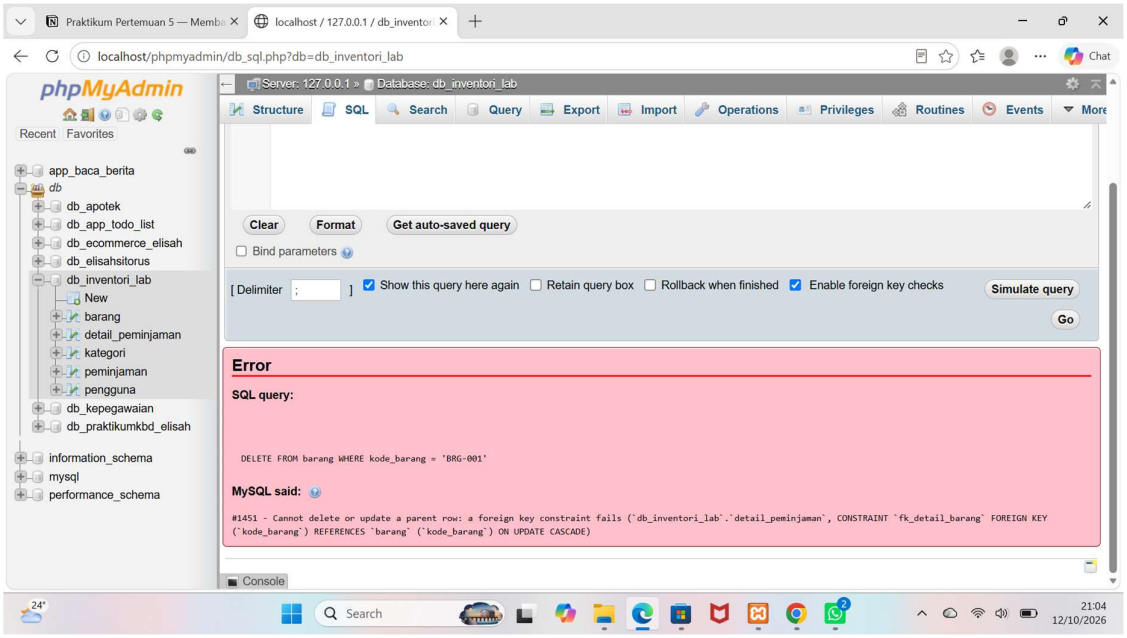
Hasil: Query ditolak. Ketentuan NOT NULL mengharuskan nama barang diisi dan tidak boleh kosong.

16. Uji Error 5: Menghapus Barang yang Masih Dipakai (Foreign Key)

Percobaan ini menghapus Keyboard USB berkode BRG-001, padahal barang tersebut masih tercatat dalam detail transaksi peminjaman.

Perintah SQL:

```
DELETE FROM barang
WHERE kode_barang = 'BRG-001';
```



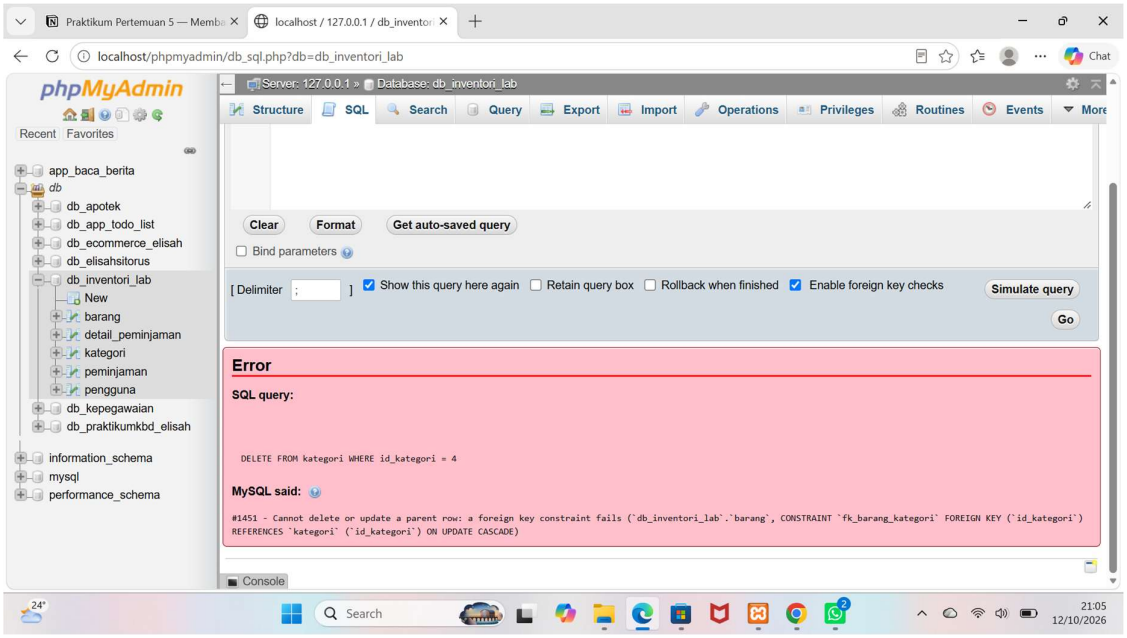
Hasil: Penghapusan ditolak agar data barang yang masih dirujuk oleh riwayat peminjaman tetap terjaga.

17. Uji Error 6: Menghapus Kategori yang Masih Dipakai Barang

Langkah ini mencoba menghapus kategori dengan id_kategori 4, yaitu Alat Ukur, meskipun kategori tersebut masih digunakan oleh data barang.

Perintah SQL:

```
DELETE FROM kategori
WHERE id_kategori = 4;
```



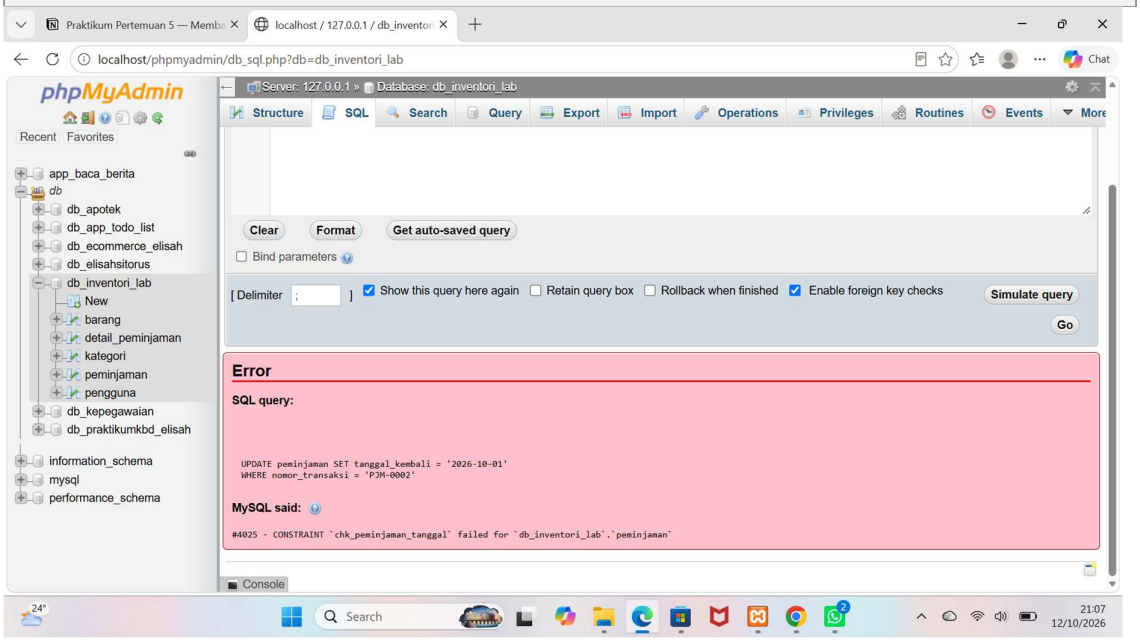
Hasil: Query gagal karena kategori masih memiliki barang yang terhubung. Barang terkait perlu dipindahkan ke kategori lain atau dihapus terlebih dahulu sesuai aturan database.

18. Uji Error 7: Pelanggaran CHECK pada Tanggal Peminjaman

Pengujian ini menetapkan tanggal kembali menjadi 2026-10-01, padahal tanggal pinjam transaksi PJM-0002 adalah 2026-10-08.

Perintah SQL:

```
UPDATE peminjaman
SET tanggal_kembali = '2026-10-01'
WHERE nomor_transaksi = 'PJM-0002';
```



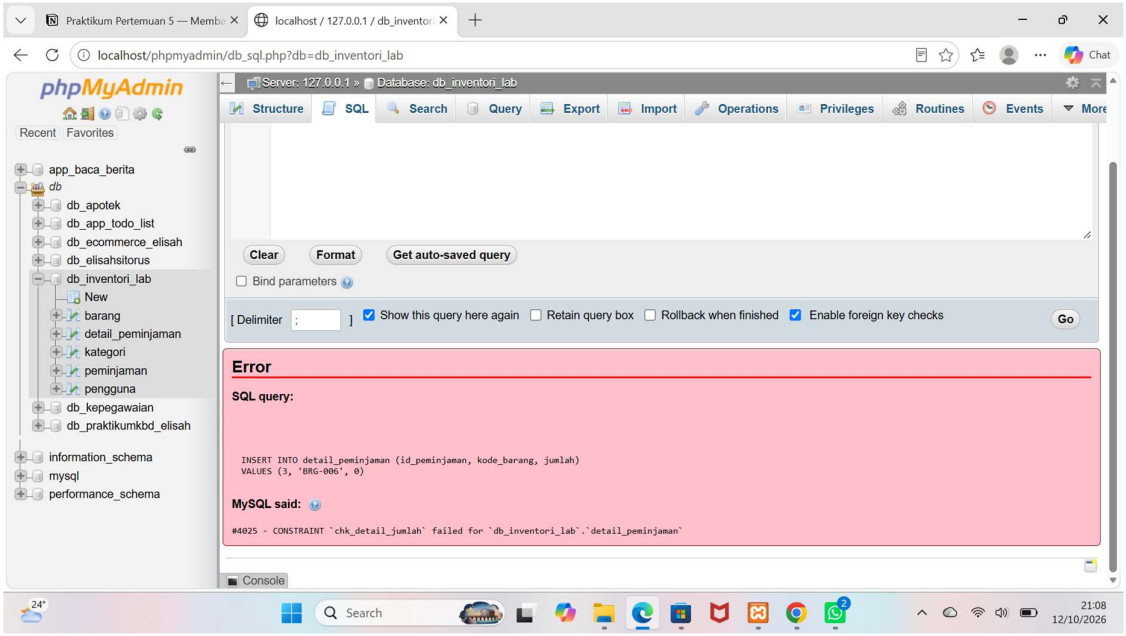
Hasil: Query ditolak karena tanggal kembali tidak boleh lebih awal daripada tanggal peminjaman, sehingga urutan tanggal tetap masuk akal.

19. Uji Error 8: Pelanggaran CHECK pada Jumlah Detail Peminjaman

Query ini mencoba menambahkan detail peminjaman dengan jumlah barang sebesar 0 unit.

Perintah SQL:

```
INSERT INTO detail_peminjaman (id_peminjaman, kode_barang, jumlah)
VALUES (3, 'BRG-006', 0);
```



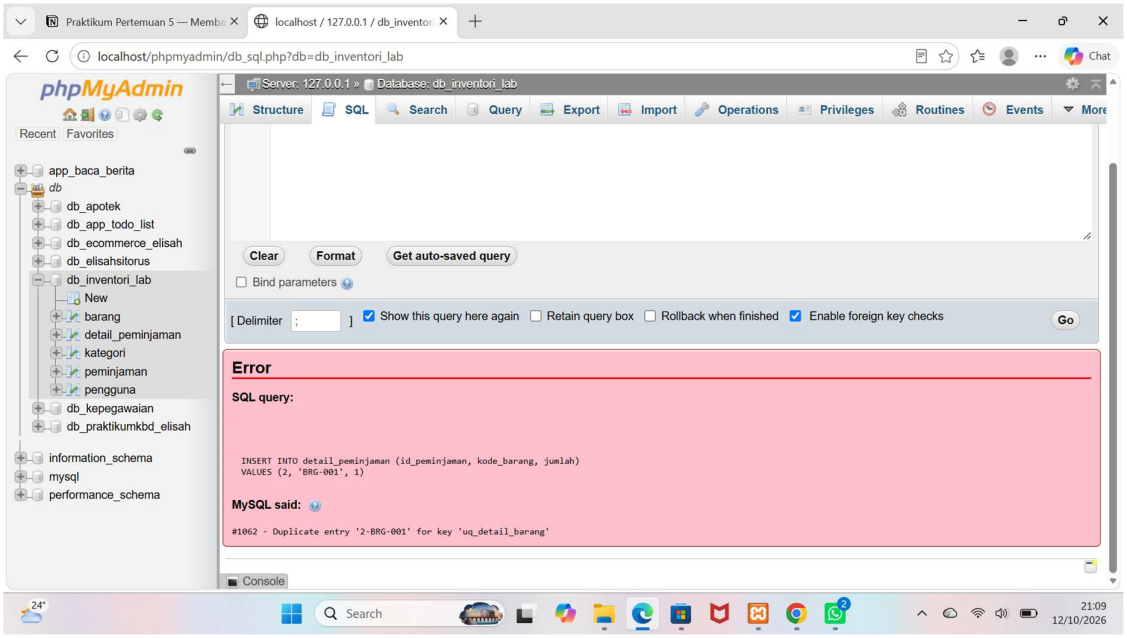
Hasil: Query gagal karena jumlah barang yang dipinjam harus memenuhi ketentuan dan tidak boleh bernilai nol.

20. Uji Error 9: Unique Key Ganda pada Detail Peminjaman

Percobaan ini memasukkan kembali barang BRG-001 (Keyboard USB) ke transaksi dengan id_peminjaman 2 (PJM-0002), meskipun barang tersebut sudah tercatat pada transaksi yang sama.

Perintah SQL:

```
INSERT INTO detail_peminjaman (id_peminjaman, kode_barang, jumlah)
VALUES (2, 'BRG-001', 1);
```



Hasil: Query ditolak. Unique key mencegah pencatatan ganda untuk barang yang sama dalam satu transaksi peminjaman.

21. Menambah Data Peminjaman Latihan beserta Detailnya

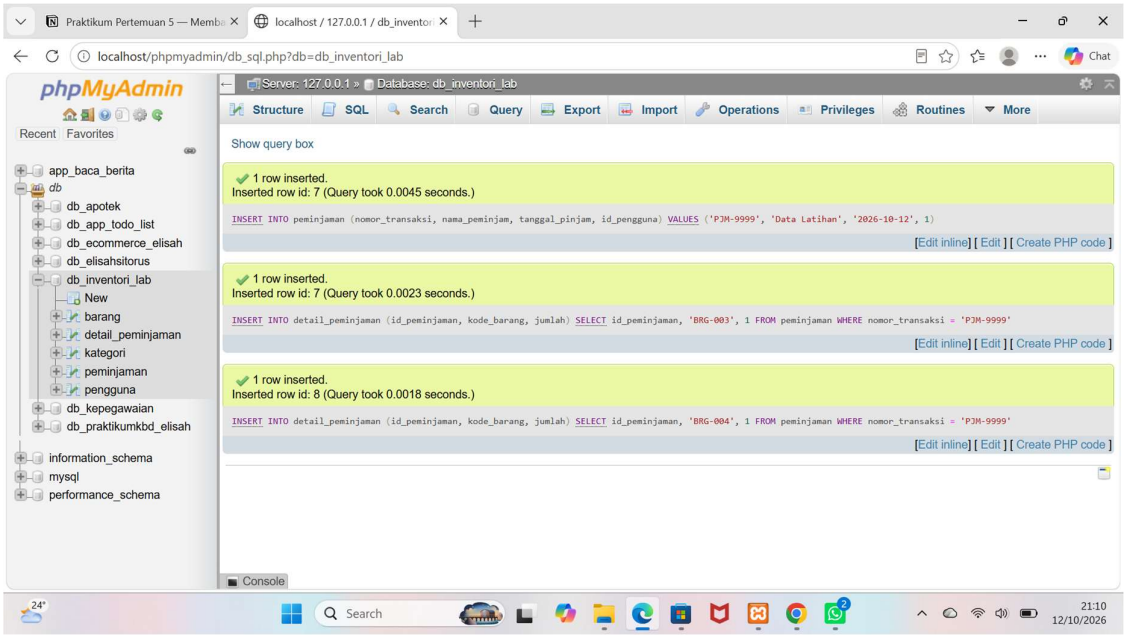
Tiga query INSERT berikut membuat transaksi percobaan bernomor PJM-9999, lalu menambahkan dua barang ke tabel detail_peminjaman untuk transaksi tersebut.

Perintah SQL:

```
INSERT INTO peminjaman (nomor_transaksi, nama_peminjam, tanggal_pinjam,
id_pengguna)
VALUES ('PJM-9999', 'Data Latihan', '2026-10-12', 1);

INSERT INTO detail_peminjaman (id_peminjaman, kode_barang, jumlah)
SELECT id_peminjaman, 'BRG-003', 1
FROM peminjaman WHERE nomor_transaksi = 'PJM-9999';

INSERT INTO detail_peminjaman (id_peminjaman, kode_barang, jumlah)
SELECT id_peminjaman, 'BRG-004', 1
FROM peminjaman WHERE nomor_transaksi = 'PJM-9999';
```



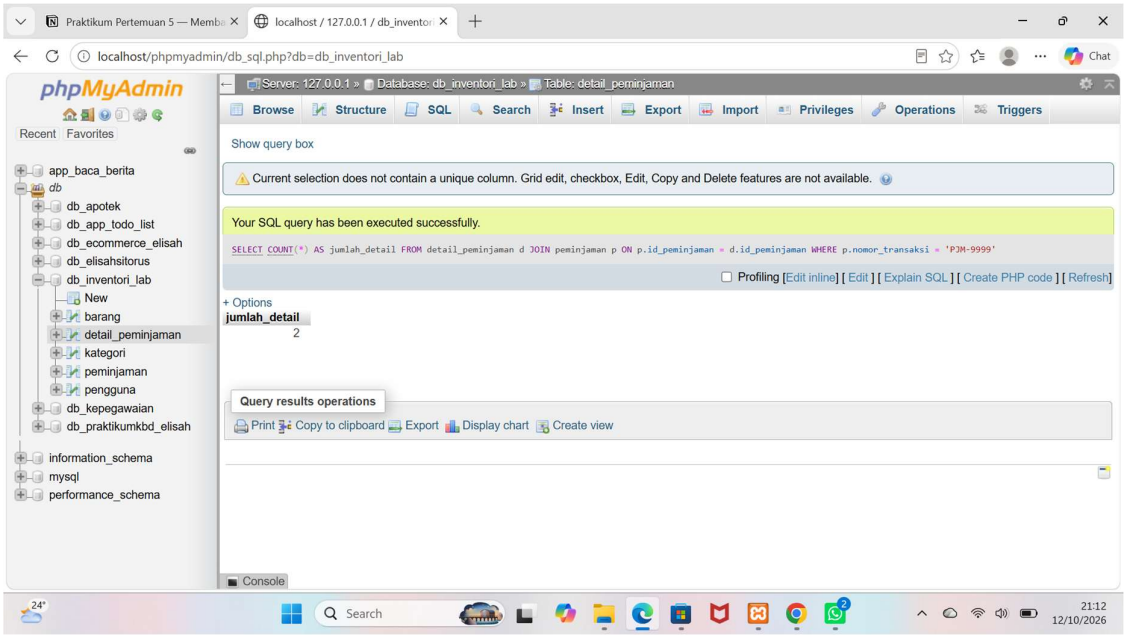
Hasil: Ketiga query berhasil dijalankan dan masing-masing menambahkan satu baris. Data percobaan ini digunakan untuk memeriksa mekanisme penghapusan berantai.

22. Menghitung Jumlah Detail Transaksi PJM-9999

Query ini menghitung jumlah baris detail yang terhubung dengan transaksi PJM-9999, sebelum transaksi tersebut dihapus.

Perintah SQL:

```
SELECT COUNT(*) AS jumlah_detail
FROM detail_peminjaman d
JOIN peminjaman p ON p.id_peminjaman = d.id_peminjaman
WHERE p.nomor_transaksi = 'PJM-9999';
```



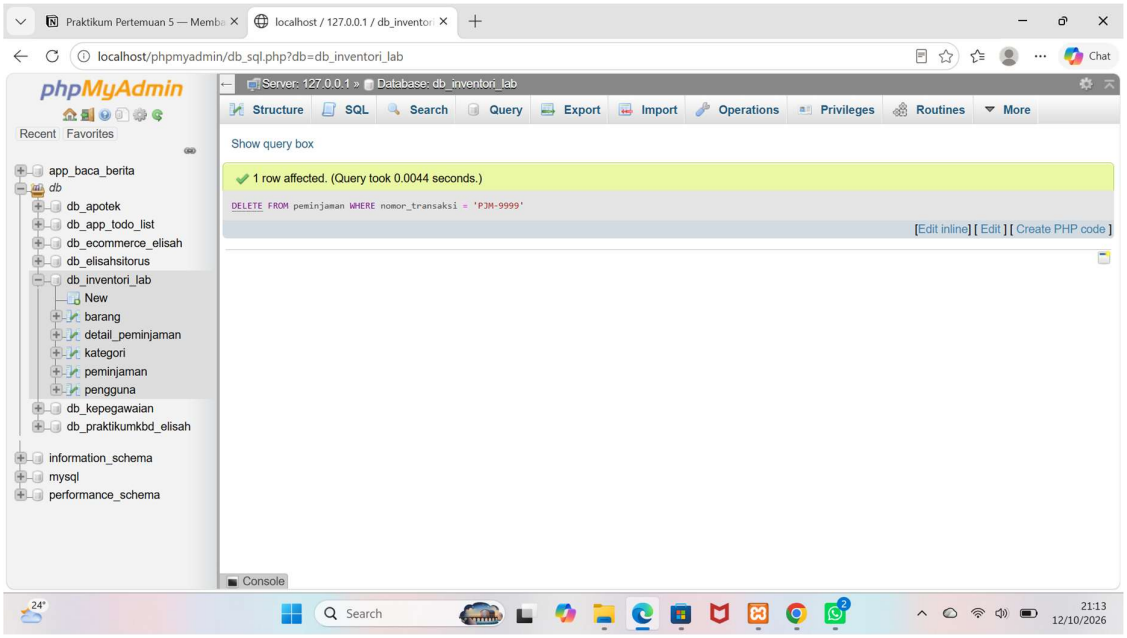
Hasil: Pemeriksaan menunjukkan ada 2 baris detail yang terdaftar pada PJM-9999 sebelum proses DELETE.

23. Menghapus Transaksi Peminjaman (DELETE pada Tabel Induk)

Transaksi percobaan PJM-9999 dihapus dari tabel peminjaman meskipun masih memiliki dua baris terkait di tabel detail_peminjaman.

Perintah SQL:

```
DELETE FROM peminjaman
WHERE nomor_transaksi = 'PJM-9999';
```



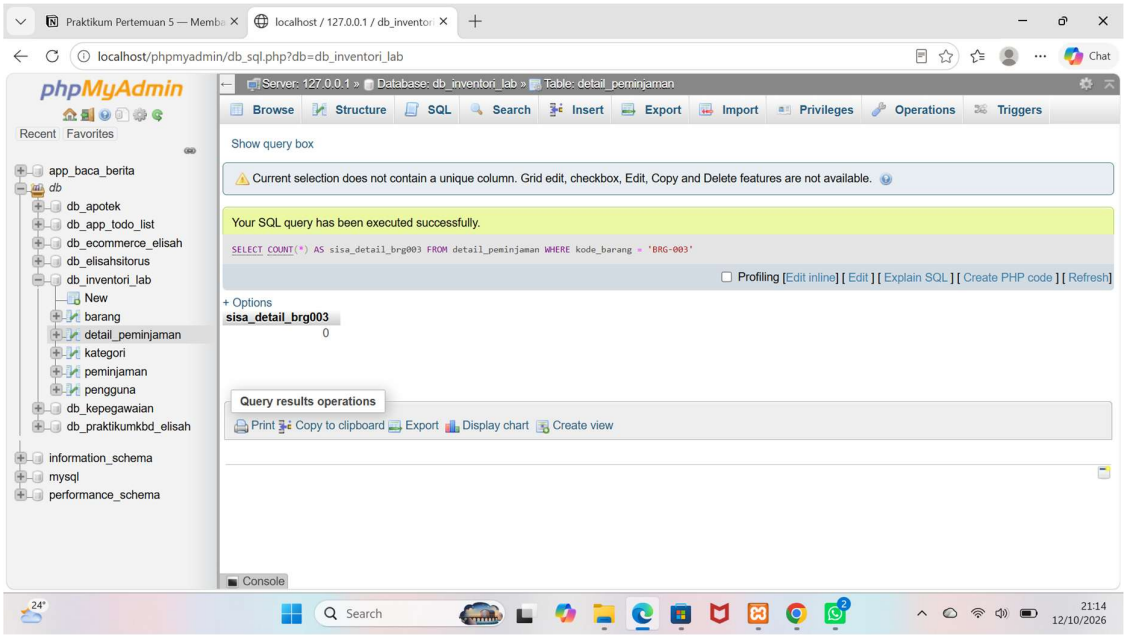
Hasil: Data transaksi PJM-9999 berhasil dihapus. Pada langkah berikutnya, dilakukan pemeriksaan untuk memastikan detail transaksinya juga terhapus.

24. Pembuktian Penghapusan Berantai (CASCADE) pada Detail

Query ini memeriksa apakah detail milik transaksi PJM-9999 masih tersimpan setelah data induknya dihapus. Kode BRG-003 digunakan sebagai sampel pemeriksaan.

Perintah SQL:

```
SELECT COUNT(*) AS sisa_detail_brg003
FROM detail_peminjaman
WHERE kode_barang = 'BRG-003';
```



Hasil: Fitur penghapusan berantai (CASCADE) berjalan sesuai rancangan. Saat transaksi induk dihapus dari tabel peminjaman, baris detail yang terkait pada tabel detail_peminjaman ikut terhapus secara otomatis.